

Comprehensive Technical Specification & Architecture Implementation

- **Gradient Reversal Layer (GRL):** In forward pass, returns input unchanged; in backward pass, multiplies gradients by $-\alpha$. Adversarially removes domain-specific/generator signatures.
- **Domain Classifier Head:** $\text{nn.Linear}(1664, 256) \rightarrow \text{GELU} \rightarrow \text{Dropout}(0.2) \rightarrow \text{Linear}(256, 2)$. Classifies dataset source/generator, encouraging the backbone to learn domain-invariant representations.

3. Model Layer & Tensor Shape Summary

Submodule / Layer	Input Shape	Output Shape	Activation / Regularization
EfficientNet-B0 Stream	[B, 3, 224, 224]	[B, 1280]	Swish / AdaptiveAvgPool
DINOv2 ViT-S/14 Stream	[B, 3, 224, 224]	[B, 384]	LayerNorm / Self-Attention
Feature Concatenation	[B, 1280] & [B, 384]	[B, 1664]	torch.cat(..., dim=1)
Layer Normalization	[B, 1664]	[B, 1664]	nn.LayerNorm(1664)
Classifier Dense 1	[B, 1664]	[B, 512]	GELU, Dropout(p=0.3)
Classifier Dense 2	[B, 512]	[B, 128]	GELU, Dropout(p=0.2)
AI Binary Logit Output	[B, 128]	[B, 1]	Linear Logit (Raw)
DANN Domain Head	[B, 1664]	[B, 2]	GRL (- α), GELU, Dropout(0.2)

4. Python Code Interface & Usage

```
from src.models import build_model

# Instantiate Ultimate Hybrid Detector
model = build_model(pretrained=True, architecture='hybrid_effnet_dinov2')

# Standard Inference Mode (Returns AI logit [B, 1]):
ai_logits = model(images) # shape: [batch_size, 1]
probs = torch.sigmoid(ai_logits)

# Multi-Task DANN Training Mode (Returns AI logit + Domain logit):
ai_logits, domain_logits = model(images, alpha=0.5, return_domain=True)
```